
1. Basic Git Commands (Foundation)

1. Initialize Repository

 Create a new Git repository

```
git init
```

 Example:

```
mkdir MyProject
```

```
cd MyProject
```

```
git init
```

2. Clone Repository

 Copy an existing repo from GitHub

```
git clone https://github.com/user/repo.git
```

 Example:

```
git clone https://github.com/dotnet/aspnetcore.git
```

3. Check Status

 Shows modified/untracked files

```
git status
```

4. Add Files (Staging Area)

 Move changes to staging

```
git add filename
```

```
git add .
```

 Example:

```
git add Program.cs
```

```
git add .
```

5. Commit Changes

👉 Save snapshot in repository

```
git commit -m "message"
```

📌 Example:

```
git commit -m "Added login API"
```

6. View Commit History

```
git log
```

```
git log --oneline
```

7. Push Changes

👉 Upload code to remote repo

```
git push origin main
```

8. Pull Changes

👉 Get latest code from remote

```
git pull origin main
```

9. Configure Git

```
git config --global user.name "Your Name"
```

```
git config --global user.email "you@example.com"
```

2. Intermediate Git Commands

1. Branching

 Create and manage branches

`git branch feature-login`

`git checkout feature-login`

OR (modern way):

`git switch -c feature-login`

2. Merge Branch

 Combine changes

`git checkout main`

`git merge feature-login`

3. Delete Branch

`git branch -d feature-login`

4. Stash Changes

 Temporarily save work

`git stash`

`git stash pop`

 Example:

`git stash save "WIP login"`

5. View Differences

`git diff`

`git diff --staged`

6. Reset Changes

👉 Undo changes

```
git reset --soft HEAD~1
```

```
git reset --hard HEAD~1
```

📌 Meaning:

- --soft → keep changes
- --hard → delete changes permanently ⚠️

7. Remove File

```
git rm file.txt
```

8. Rename File

```
git mv old.txt new.txt
```

🚀 3. Advanced Git Commands

1. Rebase (Important for Clean History)

👉 Reapply commits on top of another branch

```
git checkout feature
```

```
git rebase main
```

📌 Use when:

- You want **linear history**
 - Avoid unnecessary merge commits
-

2. Interactive Rebase

👉 Edit commit history

```
git rebase -i HEAD~3
```

You can:

- squash commits
 - rename messages
 - delete commits
-

3. Cherry Pick

👉 Pick specific commit

```
git cherry-pick commit_id
```

📌 Example:

```
git cherry-pick a1b2c3d
```

4. Reflog (Recovery Tool)

👉 Track all HEAD movements

```
git reflog
```

📌 Useful to recover deleted commits

5. Tagging

👉 Mark release versions

```
git tag v1.0
```

```
git push origin v1.0
```

6. Clean Untracked Files

`git clean -f`

7. Blame

👉 Check who changed code

`git blame file.cs`

8. Bisect (Debugging)

👉 Find bug using binary search

`git bisect start`

`git bisect bad`

`git bisect good commit_id`

9. Submodules

👉 Add external repo inside your repo

`git submodule add https://github.com/user/lib.git`

10. Worktree

👉 Multiple working directories

`git worktree add ../new-branch feature-branch`

🧠 Key Concepts Summary

Concept	Meaning
Working Directory	Your files
Staging Area	Prepared changes

Concept	Meaning
Repository	Saved history
Commit	Snapshot
Branch	Parallel development
Merge	Combine branches
Rebase	Rewrite history

⚠ Best Practices

- ☒ Commit frequently (small commits)
 - ☒ Use meaningful messages
 - ☒ Prefer **rebase over merge** (in clean workflows)
 - ☐ ⚠ Avoid git reset --hard unless sure
 - ☒ Pull before push
-